

Code Assessment of the Aave Stable Vaults

July 03, 2026

Produced for



by



Contents

1	Executive Summary	3
2	Assessment Overview	5
3	Limitations and use of report	22
4	Terminology	23
5	Open Findings	24
6	Resolved Findings	25
7	Informational	33
8	Notes	35

1 Executive Summary

Dear all,

Thank you for trusting us to help Aave Labs with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Aave Stable Vaults according to [Scope](#) to support you in forming an opinion on their security risks.

Aave Labs implements a cross-chain semi-fixed-rate stablecoin vault where users deposit stablecoins on the Accounting Chain (Arbitrum) and earn predetermined per-second interest rates, while protocol managers deploy those assets into ERC-4626 yield strategies across Accounting and Earning Chains. Withdrawals are two-phase: users first request IOUs representing their claim, then execute the withdrawal by burning IOUs for underlying assets, subject to configurable fees.

The most critical subjects covered in our audit are functional correctness, access control, asset solvency, external integrations, and defense in depth against trusted but potentially compromised components. Security regarding all the aforementioned subjects is high.

The general subjects covered are documentation and code quality. Both documentation and code quality are satisfactory.

In summary, we find that the codebase provides a good level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but do not replace other vital measures to secure a project.

Note that ChainSecurity had audited a prior version of the Aave Stable Vaults codebase. Prior findings can be found in the previous report.

The following sections will give an overview of the system, our methodology, the issues uncovered and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	4
• Code Corrected	2
• Specification Changed	1
• Acknowledged	1

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The following contracts are in scope:

```
src/bridging/BaseBridgeAdapter.sol
src/bridging/adi/AdiAdapter.sol
src/bridging/ccip/CcipAdapter.sol
src/core/Allocator.sol
src/core/BaseChainGateway.sol
src/core/LocalBalanceAggregator.sol
src/core/accounting/AccountingChainGateway.sol
src/core/accounting/FundsHandler.sol
src/core/accounting/StableVault.sol
src/core/earning/EarningChainGateway.sol
src/core/ious/IouToken.sol
src/core/ious/IouTokenManager.sol
src/interfaces/IAccountingChainGateway.sol
src/interfaces/IAdiBridgeAdapter.sol
src/interfaces/IAllocator.sol
src/interfaces/IAssetRegistry.sol
src/interfaces/IBridgeAdapter.sol
src/interfaces/ICcipBridgeAdapter.sol
src/interfaces/IChainBalanceOracle.sol
src/interfaces/IChainBalanceOracleAdapter.sol
src/interfaces/IChainGateway.sol
src/interfaces/IDepositPolicy.sol
src/interfaces/IEarningChainGateway.sol
src/interfaces/IEarningChainStateProvider.sol
src/interfaces/IFundsBridgingPolicy.sol
src/interfaces/IFundsHandler.sol
src/interfaces/IIouToken.sol
src/interfaces/IIouTokenManager.sol
src/interfaces/IMintableBurnableIERC20.sol
src/interfaces/IMulticall.sol
src/interfaces/IPolicyRegistry.sol
src/interfaces/IPriceOracle.sol
src/interfaces/IPriceOracleAdapter.sol
src/interfaces/IRebalancePolicy.sol
src/interfaces/IRescuableNative.sol
src/interfaces/IRescuableToken.sol
src/interfaces/ISlippageCoverageVault.sol
src/interfaces/IStableVault.sol
src/interfaces/ISwapper.sol
src/interfaces/ITransferHelper.sol
src/interfaces/IWithdrawalExecutionPolicy.sol
src/interfaces/IWithdrawalRequestPolicy.sol
src/libraries/AssetLib.sol
```

```

src/libraries/MathLib.sol
src/libraries/RateLimitBucketLib.sol
src/misc/Multicall.sol
src/misc/RescuableNative.sol
src/misc/RescuableToken.sol
src/misc/TransferHelperClient.sol
src/oracles/balance/ChainBalanceOracle.sol
src/oracles/balance/ChainlinkChainBalanceOracleAdapter.sol
src/oracles/balance/ChainlinkL2ChainBalanceOracleAdapter.sol
src/oracles/common/L2ChainlinkOracleAdapter.sol
src/oracles/price/ChainlinkL2PriceOracleAdapter.sol
src/oracles/price/ChainlinkPriceOracleAdapter.sol
src/oracles/price/PriceOracle.sol
src/periphery/AssetRegistry.sol
src/periphery/EarningChainStateProvider.sol
src/periphery/EarningChainStateSchemaV1.sol
src/periphery/OwnedMulticall.sol
src/periphery/PolicyRegistry.sol
src/periphery/SlippageCoverageVault.sol
src/periphery/Swapper.sol
src/periphery/TransferHelper.sol
src/policies/DepositPolicy.sol
src/policies/FundsBridgingPolicy.sol
src/policies/WithdrawalExecutionPolicy.sol
src/policies/base/GlobalRateLimitedPolicy.sol
src/types/Constants.sol
src/types/Errors.sol

```

Moreover, the AccessManager role configuration scripts were reviewed and verified against the Stable Vault Roles Spreadsheet shared by the client. This covers the relevant code in the following files:

```

script/base/RolesConfig.sol
script/base/AccessManagerBaseSetup.sol
script/base/AccessManagerAccountingChainSetup.sol
script/base/AccessManagerEarningChainSetup.sol

```

The assessment was performed on the source code files inside the Aave Stable Vaults repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received. Note that **Version 2** of this review does not cover the access control configuration; this was rechecked in **Version 3** and **Version 4**.

V	Date	Commit Hash	Note
1	12 May 2026	1c179e978c9deb5517686d35915ef54c545745aa	Initial Version
2	04 June 2026	3afa6b4a5dbb9381cb540fd4a7399bf5c021e7c6	After Intermediate Report
3	05 June 2026	b779727d1fc9ee8c9ac6c0c3ff15e3cf89907f7d	Fixes for Findings
4	17 June 2026	b832e9260593709135049f845e7b9ceac7b26918	Global Rate Limits

For the solidity smart contracts, the compiler version 0.8.28 and evm version cancun were chosen.

2.1.1 Excluded from scope

Any file not explicitly listed in scope is excluded from this review. In particular:

- Deployment and configuration scripts
- Test files
- Third-party dependencies (OpenZeppelin, Chainlink contracts, a.DI bridge)

Additionally, gas optimizations are excluded from this assessment.

2.2 Assumptions

This assessment assumes the validity of the assumptions documented by the client in the StableVault specifications.

Issues arising from violations of these assumptions are outside the scope of this audit. Where the documentation specifies a defensive mechanism (for example, treating stale Chainlink data as 0 in balance aggregation), the assessment verifies that the mechanism is correctly and completely implemented in code.

This assessment also evaluates whether the code's failure handling at trust boundaries is consistent with the stated trust level. This applies primarily to fund management flows: at any site where a trusted or partially trusted dependency supplies data or initiates a fund management action, bad inputs or states detectable within the smart contract are expected to be blocked. Components labeled "trusted" may still warrant on-chain safeguards against malfunction. This must be detectable within the smart contract's function and actions that are indistinguishable from honest activity of these roles / dependencies within the smart contract are not in scope of this. Gaps between stated or expected defenses and their on-chain handling are in scope. The underlying trust statements themselves are not. Functions whose primary protection is a timelock, and functions that do not affect core system flow are excluded from this defense in depth assessment.

Key assumptions include:

(#1) Admin actions are gated via multisig with timelock: High-risk functions guarded by the restricted modifier are accessible only to multisig wallets and, in most if not all cases, carry an execution timelock of at least 7 days. Guardian revocations of compromised roles are immediate (zero delay).

(#2) Common USD denomination and price oracle depeg handling: All assets deposited into an instance of the StableVault share a common denomination (USD). Price oracles validate prices on deposit and on swap output. On depeg towards zero, deposits of the asset revert, swaps outputting the asset revert, and the asset's contribution to balance aggregation is reduced proportionally to its price. Risk managers may additionally mark the asset as distrusted to exclude it from trusted assets accounting. Users may still choose to redeem IOUs into a distrusted asset, bearing any market value loss. The risk that a user frontruns a `distrustAsset()` or `disableDeposit()` transaction is acknowledged. The price oracle is the primary defense. If an asset depegs after a successful deposit, the user's position is not marked down. It remains 1:1 with the registered token quantity deposited. The system accepts only ERC-20 tokens registered in the AssetRegistry as deposit assets (not IOU tokens, not native asset, not ERC-4626 strategy shares).

(#3) Standard ERC-20 tokens: Deposited tokens have no fee-on-transfer behavior, do not rebase, do not invoke transfer callbacks (ERC-777, ERC-1362), and have at most 18 decimals. The system relies on SafeERC20 to handle non-standard return values such as those of USDT.

(#4) ERC-4626 strategies are audited and trusted: Whitelisted strategies are audited and trusted, charge no withdrawal fees (so that `previewRedeem` accounting is accurate), are not susceptible to inflation attacks, and are continuously monitored off-chain. If a strategy is compromised, malfunctions, or otherwise becomes unsafe, deposits to it are disabled immediately via a zero timelock single signature

hotkey, and deallocations and any further actions are coordinated by the responsible parties. The handling of such an event and its impact are out of scope.

(#5, #9, #25) Bridge integrity, finality, and out-of-order delivery: Arbitrary data sent through the system's bridge adapter contracts is not tampered with by the bridge provider, and messages are delivered only after source chain block finalization. The system does not depend on ordered delivery. Adapters, gateways, and FundsHandler ingest messages safely out of order, with block number validation against the chain balance oracle ensuring consistency between IOU burn or fund return messages and the oracle's view of Earning Chain balance. Aave Delivery Infrastructure is responsible for replay protection with respect to envelopes.

(#7, #8) Single instance topology: A given set of Earning Chain contracts is used by exactly one Accounting Chain instance, with its own dedicated IOU Token Manager and IOU Token on each chain. Reuse across instances breaks system invariants and is not in scope.

(#10) Cross chain configuration consistency: The Admin maintains configuration consistency between the Accounting Chain and every Earning Chain, including paired bridge adapters, mutual asset support on every bridged route, correctly wired IOU Token Manager and IOU Token addresses with the `IS_ACCOUNTING_CHAIN` flag set appropriately, and oracle adapters kept in sync.

(#12) Chainlink infrastructure trusted and highly available: Chainlink price feeds and the custom chain balance feed are trusted and highly available. Total outage is treated as a corner case for which the system fails conservatively. Stale prices contribute 0 to balance aggregation, and stale chain balances are treated as 0. Verifying that this conservative behavior is correctly implemented at every aggregation and gating point is in scope.

(#14) Earning Chain removal by a trusted actor: Removing an Earning Chain ID from FundsHandler is performed by a trusted actor only when deemed necessary. Findings premised on adversarial removals are out of scope.

(#17) Manager revocation within ~1 hour: Compromised or malicious Manager (Rebalancer) accounts are revoked by guardians within approximately one hour of detection, without a timelock, via off-chain monitoring. This bound is the basis for accepting that a Manager can, within the detection window, repeatedly rebalance the Allocator (incurring slippage on ERC-4626 deposits and withdrawals), or halt yield accrual by deallocating from all strategies.

(#19) Array length mismatch reverts via OOB read: When multiple arrays are passed as inputs to the same function, explicit length checks are not required. Out of bounds reads cause the transaction to revert. Findings consisting solely of "missing array length check" are out of scope.

(#22) Conservative underestimation is acceptable: Underestimating prices or balances from stale oracle feeds is acceptable even when it temporarily disallows withdrawal of earned interest or claiming of surplus interest. The system is biased toward protecting original deposits over enabling withdrawal of profits.

(#23, #26) Off-chain operational monitoring: Off-chain systems handle (i) re-execution of dropped bridge messages when on-chain ingestion fails, and (ii) re-quoting of withdrawal fee discounts when users transfer shares between addresses to attempt to game discount eligibility. Re-execution of messages is done via the CCIP explorer for CCIP, and via `AdiAdapter.retryTransaction()` or `retryEnvelope()` for aDI.

(#24) Default SubVault availability: The default SubVault in the StableVault is always active or has an available slot when a user deposit is routed through it.

2.3 System Overview

Aave Stable Vaults was previously audited by ChainSecurity. This system overview describes the version of the contracts under the current engagement, as defined in the [Assessment Overview](#).



Aave Labs offers Aave Stable Vaults, a cross-chain semi-fixed-rate stablecoin vault. Users deposit stablecoins on the Accounting Chain (Arbitrum) and earn a predetermined per-second interest rate, while protocol managers deploy those assets into ERC-4626 yield strategies on the Accounting Chain as well as to Earning Chains, with Ethereum mainnet as the initial Earning Chain. Withdrawals are two-phase: users first request a withdrawal that converts their vault position into IOU tokens, then execute the withdrawal by burning IOUs for underlying assets, subject to a configurable fee. The key actors are depositors, IOU holders, protocol managers who rebalance and bridge funds, a governance layer backed by OpenZeppelin AccessManager with timelocked and immediate execution tiers, Chainlink oracle infrastructure that relays price and balance data across chains, and Chainlink CCIP together with Aave Delivery Infrastructure (a.DI) for cross-chain messaging.

2.3.1 StableVault

StableVault is the user-facing entry point on the Accounting Chain. It implements a semi-fixed-rate vault where each interest rate tier is modeled as a *sub-vault* with its own per-second conversion rate and share supply. When a user deposits, their assets are routed through FundsHandler into the Allocator, where they are held idle until a manager deploys them into yield strategies. In return, the user receives shares in their assigned sub-vault. The conversion rate accrues continuously: $\text{conversionRate} *= \text{perSecondRate} \wedge \text{elapsed}$. A user's reported balance in RAY terms is $\max(\text{userShares} * \text{conversionRate}, \text{originalDepositRay})$: the share-derived value is floored at the user's original deposit so that rounding can never make the reported balance fall below the principal the system guarantees. Each user holds at most one position in one sub-vault, where governance can move users between sub-vaults or change sub-vault rates.

Deposits and withdrawals pass optional policyData to policy contracts resolved from PolicyRegistry. On deposit, DepositPolicy enforces per-asset rate limits and a global cross-asset cap. On withdrawal request, WithdrawalRequestPolicy may gate the request. On withdrawal execution, WithdrawalExecutionPolicy applies the withdrawal fee and enforces a global redemption rate limit. The two-step withdrawals work as follows. In requestWithdrawal(), shares are burned and the vault computes how much of the withdrawal is *guaranteed* (backed by the user's original deposit principal) versus *interest* (which must be covered by the system's surplus assets over obligations). The function enforces a global solvency check, where the interest portion of the request must not exceed $\text{totalAssets} - \text{guaranteedObligations}$, where totalAssets aggregates local Allocator balances (trusted assets only) and the latest Earning Chain balance snapshots from the ChainBalanceOracle. Upon passing the check, IOU tokens are minted to the user. In executeWithdrawal(), the user burns IOUs, where withdrawal fees are applied through WithdrawalExecutionPolicy. Fees are taken in the form of IOU tokens and burned, reducing the system's guaranteed obligations. The net amount is withdrawn from the Allocator through FundsHandler and transferred to the user.

Withdrawals consume the guaranteed (principal) portion of a user's balance first. The amount exceeding the original deposit is treated as interest and subject to the solvency check. Users can always request IOUs corresponding to their original deposits. However, when the system's surplus is insufficient to cover all accrued interest, interest withdrawals follow first-come-first-served order. Earlier withdrawers claim their interest as IOUs, while later users may be limited to their guaranteed portion.

Similarly, if the vault becomes underfunded, IOU redemption in executeWithdrawal() and EarningChainGateway.exchangeIouTokens() is first-come-first-served with respect to the assets available in the (local) Allocator.

The vault also supports transfer() and transferAll() to move positions between users (accounting-only, no asset movement), and claimSurplusInterest() which lets governance withdraw earned yield above total obligations to a treasury address.

The contract provides the following functions:

- deposit(user, asset, amount, policyData): Permissionless. Deposits a whitelisted stablecoin for the specified user. Applies DepositPolicy rate limits, validates the asset price via PriceOracle.validatePrice(), routes funds through FundsHandler into the Allocator,

and mints sub-vault shares. Reverts if the asset's price is below the minimum valid price, if deposits are disabled for that asset, or if the deposit policy rejects the amount.

- `requestWithdrawal(user, requestedAmountInRay, policyData)`: Only callable by the user themselves. Burns sub-vault shares and mints IOU tokens. Passing 0 triggers a full withdrawal. Enforces the global solvency invariant before minting IOUs. Applies `WithdrawalRequestPolicy` if configured.
- `executeWithdrawal(user, assetOut, minAmountOut, iouAmountRay, policyData)`: Only callable by the user themselves. Burns IOUs, applies the withdrawal fee via `WithdrawalExecutionPolicy`, withdraws assets from `Allocator`, and transfers them to the user.
- `transfer(to, amountRay)`: Transfers a portion of the caller's vault position to another user. Accounting-only. `amountRay` is capped at the value the sender's shares actually back (`shares * conversionRate`, without the original-deposit floor). A request above that reverts and the sender is expected to use `transferAll()` instead. The sender's principal (`originalDepositRay`) is decremented first, up to the transferred amount. Any excess above the sender's remaining principal is treated as interest and does not contribute to the recipient's principal.
- `transferAll(to)`: Transfers the caller's entire position (shares and original deposit tracking) to another user. Accounting-only.
- `setUserRate(userRateData[])`: Restricted (immediate). Moves users to sub-vaults matching new per-second rates, creating sub-vaults if needed.
- `setSubVaultRate(subVaultId, newPerSecondRate)`: Restricted (4-hour delay). Changes the interest rate of an existing sub-vault. Accrues before updating.
- `setDefaultSubVault(perSecondRate)`: Restricted (24-hour delay). Sets the default sub-vault assigned to new depositors.
- `claimSurplusInterest/assets[], amounts[]`: Restricted (7-day delay). Withdraws assets from the `Allocator` up to the surplus above total obligations, and sends them to the treasury.
- `setTreasury(treasury)`: Restricted (7-day delay). Sets the treasury address for surplus claims.

2.3.2 FundsHandler

`FundsHandler` acts as the intermediary between the `StableVault` and the `Allocator` on the Accounting Chain. It routes deposit and withdrawal requests to the local `Allocator` and computes the system-wide aggregated balance by summing the local `Allocator` balance (via `LocalBalanceAggregator`, using only trusted assets) with Earning Chain balance snapshots from the `ChainBalanceOracle`. Stale Earning Chain balances are treated as zero, deliberately under-estimating the total asset value.

The contract also manages the set of registered Earning Chain IDs and provides the `pushFundsToChain()` function for managers to bridge assets from the Accounting Chain to an Earning Chain via the `AccountingChainGateway`. Before bridging, `FundsBridgingPolicy` is looked up from `PolicyRegistry` and applied to enforce per-route (asset, chain, adapter) transfer rate limits and a global cross-route cap. When assets arrive from an Earning Chain (via the gateway's `fundsArrivedFromChainCallback`), they are deposited into the local `Allocator`.

The contract provides the following functions:

- `processDeposit(asset, amount)`: Only callable by `StableVault`. Deposits assets into the `Allocator`.
- `processWithdrawal(asset, amount)`: Only callable by `StableVault`. Withdraws assets from the `Allocator`.
- `getAggregatedBalance()`: View. Returns the total system balance in RAY. Computes the local `Allocator` balance (trusted assets only) plus all non-stale Earning Chain balances.



- `pushFundsToChain(asset, amount, chainId, bridgeAdapter, gasLimit, bridgedata, policyData)`: Restricted (immediate). Applies `FundsBridgingPolicy`, withdraws assets from the Allocator, and bridges them to a registered Earning Chain.
- `fundsArrivedFromChainCallback(asset, amount)`: Only callable by the `AccountingChainGateway`. Deposits received assets into the Allocator.
- `addEarningChain(chainId)` / `removeEarningChain(chainId)`: Restricted (7-day delay). Manages the set of Earning Chains whose balances are included in the aggregated balance.

2.3.3 Allocator

Allocator manages the deployment of stablecoin assets into ERC-4626 yield strategies. Each registered asset can have up to `MAX_STRATEGIES_PER_ASSET` strategies. User deposits arrive via `deposit()`. On withdrawal, a cascading logic first consumes idle balance held directly by the Allocator, then iterates through strategies in the withdrawal queue order (set by a manager) until the requested amount is fulfilled.

Strategy deposits call `IERC4626.deposit()` inside a `try/catch` and verify that the deposited amount is within a fixed slippage tolerance (`STRATEGY_DEPOSIT_SLIPPAGE_TOLERANCE`) of the value reported by `previewRedeem()` on the received shares. The `rebalance` function allows managers to execute an ordered sequence of deallocations, swaps (via an external Swapper), and allocations in a single transaction.

Strategies can be individually marked as trusted or untrusted via `trustStrategy()` and `distrustStrategy()`. Balance aggregation used for solvency checks (`LocalBalanceAggregator`) includes only trusted strategies, while general balance queries include all strategies.

Deposits into and withdrawals from the Allocator are restricted to a single whitelisted `DEPOSITOR` and `WITHDRAWER` address respectively, set immutably at construction time (`FundsHandler` on the `Accounting Chain`, `EarningChainGateway` on `Earning Chains`).

The contract provides the following functions:

- `deposit(asset, amount)`: Only callable by the `DEPOSITOR`. Pulls the asset from the caller via `TransferHelper` and leaves it idle.
- `withdraw(asset, amount)`: Only callable by the `WITHDRAWER`. Reverts if the full amount cannot be sourced.
- `topUp(asset, amount)`: Restricted (immediate). Pulls a registered asset from the caller into the Allocator as idle balance, improving solvency.
- `rebalance(params[], policyData)`: Restricted (immediate). Executes ordered deallocations, swaps, and allocations. Applies `RebalancePolicy` post-execution.
- `setWithdrawalQueue(asset, newWithdrawalQueue[])`: Restricted (immediate). Sets the ordered list of strategies for the withdrawal cascade. Must be a strict permutation of the registered strategies for that asset.
- `addStrategy(asset, strategy)`: Restricted (7-day delay). Registers a new ERC-4626 strategy for an asset.
- `removeStrategy(strategy)`: Restricted (immediate). Removes a strategy. Requires the strategy to be trusted and to hold a zero balance.
- `trustStrategy(strategy)`: Restricted (7-day delay). Marks a strategy as trusted for inclusion in solvency balance aggregation.
- `distrustStrategy(strategy)`: Restricted (immediate). Excludes a strategy from solvency balance aggregation.
- `enableDepositsToStrategy(strategy)` / `disableDepositsToStrategy(strategy)`: Restricted (7-day delay / immediate respectively). Toggles deposit permission per strategy.

2.3.4 AccountingChainGateway

AccountingChainGateway extends BaseChainGateway and serves as the cross-chain messaging hub on the Accounting Chain. It handles three inbound message types from Earning Chains:

- **BRIDGE_IOU_TOKEN**: Releases locked IOUs to a recipient via the IouTokenManager.
- **BURN_IOU_TOKEN**: Burns locked IOUs to synchronize obligations after an Earning Chain IOU exchange.
- **RETURN_FUNDS**: Accompanies assets bridged back from an Earning Chain to the Accounting Chain.

For outbound communication, the Accounting Chain gateway sends asset-bearing messages to Earning Chains on behalf of FundsHandler.

The `_validateInboundMessageBlockNumber()` check is a consistency mechanism. It rejects inbound **BURN_IOU_TOKEN** and **RETURN_FUNDS** messages whose source block number is newer than the latest oracle snapshot, preventing the Accounting Chain from decrementing obligations or accepting funds based on state that its balance oracle has not yet captured. Moreover, outbound fund pushes are blocked if the target chain's balance snapshot is stale.

The contract provides the following functions:

- `sendPushFundsToChainMessage(asset, amount, targetChainId, bridgeAdapter, feePayer, gasLimit, bridgeData)`: Only callable by FundsHandler. Validates the target chain balance snapshot is not stale, then sends assets to an Earning Chain via the specified bridge adapter.

2.3.5 EarningChainGateway

EarningChainGateway extends BaseChainGateway and is deployed on each Earning Chain. It communicates with the Accounting Chain. Its primary user-facing function is `exchangeIouTokens`, which allows IOU holders to burn their IOUs and receive underlying stablecoin assets from the local Allocator, subject to the withdrawal fee applied by `WithdrawalExecutionPolicy`. This enables users to exit the system on any Earning Chain that holds liquidity, not just on the Accounting Chain.

When assets are received from the Accounting Chain, they are deposited into the local Allocator using `deposit()`. Managers can return assets to the Accounting Chain via `pushFundsToAccountingChain()`, which applies `FundsBridgingPolicy` before bridging. On IOU exchange, the gateway sends a **BURN_IOU_TOKEN** message to the Accounting Chain.

The contract provides the following functions:

- `exchangeIouTokens(iouTokenAmountRay, assetOut, minAmountOut, receiver, bridgeAdapter, gasLimit, bridgeData, policyData)`: Permissionless. Burns the caller's IOUs, applies withdrawal execution policy, withdraws assets from Allocator, sends a **BURN_IOU_TOKEN** message to the Accounting Chain, and transfers assets to the specified receiver.
- `pushFundsToAccountingChain(asset, amount, bridgeAdapter, gasLimit, bridgeData, policyData)`: Restricted (immediate). Applies `FundsBridgingPolicy`, withdraws from the local Allocator, and bridges assets plus a **RETURN_FUNDS** message to the Accounting Chain.
- `getAggregatedBalance()`: View. Returns the local Allocator balance in RAY (trusted assets only).

2.3.6 BaseChainGateway

BaseChainGateway is the abstract base for both gateway contracts. It manages bridge adapter whitelisting across two independent registries: *funds* adapters, whitelisted per asset and destination chain ID, and *data-only* adapters, tracked per destination chain ID with a per-adapter mode (NOT_SUPPORTED, SEND_AND_RECEIVE, RECEIVE_ONLY). Data-only adapters use a two-step removal. Removal is first initiated, which moves the adapter to RECEIVE_ONLY. At this point, the adapter can still receive in-flight messages but can no longer send. Moreover, the last send-enabled data-only adapter for a chain cannot be removed. The `receiveMessage()` function implements a split path. Fund reception (token transfers from a bridge) does not require the calling adapter to be whitelisted, while data processing (cross-chain messages) strictly requires the calling adapter to be a registered data-only adapter able to receive (SEND_AND_RECEIVE or RECEIVE_ONLY).

The contract provides the following functions:

- `addFundsBridgeAdapter(asset, chainId, adapter)`: Restricted (7-day delay). Whitelists a new funds bridge adapter for the given asset and chain.
- `removeFundsBridgeAdapter(asset, chainId, adapter)`: Restricted (immediate). Removes a funds bridge adapter from the whitelist.
- `addDataOnlyBridgeAdapter(chainId, adapter)`: Restricted (7-day delay). Registers a data-only bridge adapter for a chain in SEND_AND_RECEIVE mode.
- `initiateDataOnlyBridgeAdapterRemoval(chainId, adapter)`: Restricted (7-day delay). Moves a data-only adapter to RECEIVE_ONLY and returns the `removalId` needed to finalize. Reverts if it is the last send-enabled data-only adapter for the chain.
- `finalizeDataOnlyBridgeAdapterRemoval(chainId, adapter, removalId)`: Restricted (7-day delay). Completes the removal of a data-only adapter that is in RECEIVE_ONLY mode.
- `sendBridgeIouTokenMessageWithFeePayer(destChainId, recipient, amount, bridgeAdapter, feePayer, gasLimit, data)`: Only callable by `IouTokenManager`. Publishes IOU bridge messages to a destination chain.
- `receiveMessage(sourceChainId, asset, amount, data)`: Callable by anyone. If the message contains data, the caller is enforced to be a registered data-only adapter (for data-only messages) or a whitelisted funds adapter for the given asset and source chain (for funds-with-data messages). Fund-only messages (no data) have no such restriction, allowing recovery of stuck tokens from removed adapters.

2.3.7 IouTokenManager

`IouTokenManager` manages the lifecycle of IOU tokens. On the Accounting Chain, IOUs are minted by `StableVault` during `requestWithdrawal()` and are locked when a user bridges them to an Earning Chain. Users can bridge their IOUs back to the Accounting Chain, at which point the locked tokens are released to the recipient. Upon receiving a BURN_IOU_TOKEN message from an Earning Chain, the locked tokens are burned.

The contract provides the following functions:

- `bridgeTokens(destinationChainId, recipient, amount, bridgeAdapter, payloadGasLimit, bridgeAdapterData)`: Permissionless. Locks IOUs (Accounting Chain) or burns IOUs (Earning Chain) and sends a bridge message.
- `mintTokens(to, amount) / burnTokens(from, amount)`: Only callable by the Vault or Gateway.
- `burnLockedTokens(amount)`: Only callable by the Gateway on the Accounting Chain. Burns previously locked IOUs.
- `releaseTokens(to, amount)`: Only callable by the Gateway on the Accounting Chain. Releases locked IOUs to a recipient.

2.3.8 IouToken

IouToken is a minimal ERC-20 token with 27 decimals (RAY precision) owned by IouTokenManager. It exposes `mint()`, `burn()`, and `lock()` functions restricted to the owner. The `lock()` function transfers tokens from a user to the manager contract, effectively escrowing them during cross-chain bridging.

2.3.9 CcipAdapter

The architecture supports multiple bridge adapters. `CcipAdapter` is the Chainlink CCIP bridge adapter, extending `BaseBridgeAdapter`. It translates between the system's chain IDs and CCIP chain selectors, constructs `EVM2AnyMessage` structs, and handles fee payment and optional native fee refunds. Inbound messages arrive via `ccipReceive()`, which validates the source sender against the registered `_destinationChainAdapterOf` mapping. Any token payload is forwarded to the `TransferHelper`, after which both funds and data are delivered to the gateway via a single `receiveMessage()` call. `CcipAdapter` supports both fund-bearing and data-only messages, though in practice a.DI is preferred for data-only bridging.

The adapter includes `replayFundsReceiving()` for recovering registered assets that were directly sent to the adapter. Registered assets stuck in the adapter can only be recovered through this replay mechanism. The function `rescueTokens` is blocked for registered assets.

The contract provides the following functions:

- `publishMessageWithFunds(destChainId, asset, amount, data, feePayer, gasLimit, bridgeAdapterData)`: Only callable by the Gateway. Constructs and sends a CCIP message with token and data payloads.
- `publishDataOnlyMessage(destChainId, data, feePayer, gasLimit, bridgeAdapterData)`: Only callable by the Gateway. Sends a data-only CCIP message without token transfer.
- `ccipReceive(message)`: Only callable by the CCIP Router. Validates source, and forwards message to the gateway for processing.
- `replayFundsReceiving(asset, amount)`: Restricted (immediate). Re-processes stuck registered assets.
- `setChainSelector(chainId, ccipChainSelector)`: Restricted (7-day delay). Maps chain IDs to CCIP selectors.
- `setDestinationChainAdapter(chainId, adapter)`: Restricted (7-day delay). Inherited from `BaseBridgeAdapter`. Sets the expected remote adapter address.

2.3.10 AdiAdapter

`AdiAdapter` is the Aave Delivery Infrastructure (a.DI) bridge adapter, extending `BaseBridgeAdapter`. It is a data-only adapter, used for cross-chain messages that carry no token transfers: `BURN_IOU_TOKEN` messages sent from `EarningChainGateway` to the Accounting Chain, and `BRIDGE_IOU_TOKEN` messages sent between chains via `IouTokenManager`.

Outbound messages are forwarded to the a.DI `CrossChainController`, which routes them through registered bridge providers to the destination chain.

a.DI tracks messages using two identifiers: an `envelopeId` that identifies the logical message and a `transactionId` that identifies one send attempt for that envelope. The adapter exposes two retry mechanisms: `retryTransaction()` re-attempts the same encoded transaction (same `transactionId`) with an updated gas limit, while `retryEnvelope()` submits a new send attempt for the same envelope (new `transactionId`, same `envelopeId`). Both retry functions are permissionless: the caller funds the retry and any excess native fee is refunded, so anyone can pay to re-attempt a stuck message. The receiving cross-chain controller tracks delivered envelopes. Inbound

messages arrive via `receiveCrossChainMessage()`, callable only by the `CrossChainController`, and are forwarded to the gateway's `receiveMessage()` handler.

The contract provides the following functions:

- `publishDataOnlyMessage(destChainId, messageData, feePayer, gasLimit, bridgeAdapterData)`: Only callable by the Gateway. Forwards the message to the `CrossChainController`.
- `quoteMessageToChain(destChainId, messageData, gasLimit)`: View. Returns the native fee estimate and per-bridge-provider quotes for delivery.
- `retryTransaction(encodedTransaction, gasLimit, bridgeAdaptersToRetry)`: Permissionless. Re-submits a failed transaction; the caller funds the retry and any excess native fee is refunded.
- `retryEnvelope(envelope, gasLimit)`: Permissionless. Re-submits a failed envelope; the caller funds the retry and any excess native fee is refunded.
- `receiveCrossChainMessage(originSender, originChainId, message, envelopeId)`: Only callable by the `CrossChainController`. Delivers a message to the gateway.
- `setDestinationChainAdapter(chainId, adapter)`: Restricted (7-day delay). Inherited from `BaseBridgeAdapter`. Sets the expected remote adapter address.

2.3.11 PriceOracle

`PriceOracle` is the registry contract that provides asset prices through pluggable adapter contracts. It enforces a ceiling of 1 RAY (1e27) on returned prices, reflecting the system's assumption that supported assets are stablecoins pegged at 1:1 with the denomination unit. An immutable `MIN_VALID_PRICE_RAY` is used by `validatePrice()` to gate deposits. Assets whose oracle price has fallen below the minimum are blocked from entering the vault.

When an adapter reports a stale price, `getPrice()` returns 0 rather than reverting, causing `LocalBalanceAggregator` to under-value the affected asset. The `validatePrice()` function, used during deposits, reverts on stale prices, preventing deposits.

The contract provides the following functions:

- `getPrice(asset)`: View. Returns the price in RAY, capped at 1 RAY. Returns 0 if stale.
- `validatePrice(asset)`: View. Reverts if the price is stale or below `MIN_VALID_PRICE_RAY`.
- `setOracleAdapterForAsset(asset, adapter)`: Restricted (7-day delay). Sets the price feed adapter for an asset.

2.3.12 ChainBalanceOracle

`ChainBalanceOracle` is the registry contract on the Accounting Chain and provides Earning Chain aggregated balance values through pluggable adapter contracts (one per Earning Chain). The adapter call is wrapped in a try/catch. If the adapter reverts, a zero-balance stale response is returned rather than propagating the failure, ensuring the consuming contracts (`FundsHandler.getAggregatedBalance()`) are not disrupted.

The contract provides the following functions:

- `getChainBalance(chainId)`: View. Returns the balance snapshot for an Earning Chain including the source chain block number and staleness status.
- `setChainBalanceOracleAdapter(chainId, adapter)`: Restricted (7-day delay). Sets the balance feed adapter for an Earning Chain.

2.3.13 Oracle Adapters

`ChainlinkPriceOracleAdapter` queries a `Chainlink AggregatorV3Interface` data feed and converts the result to RAY precision. It marks the response as stale if the feed has not been updated within the last `HEARTBEAT + 90s`.

`ChainlinkChainBalanceOracleAdapter` queries a `Chainlink Bundle Aggregator` that publishes `EarningChainStateProvider.getState()` data. It validates the schema version and chain ID within the snapshot, and marks the response as stale using the same heartbeat pattern.

Both adapters have L2 variants (`ChainlinkL2PriceOracleAdapter`, `ChainlinkL2ChainBalanceOracleAdapter`) that extend the base adapters with a `Chainlink L2 Sequencer Uptime Feed` check. If the sequencer is down or has been up for less than the 2-hour grace period, all responses are marked as stale.

`EarningChainStateProvider` is deployed on each Earning Chain and exposes the local Allocator's aggregated balance (via `EarningChainGateway.getAggregatedBalance()`). A Chainlink oracle network reads this on-chain and publishes it to the Accounting Chain's bundle aggregator.

2.3.14 Periphery Contracts

`AssetRegistry` manages registered assets and their configuration flags (`depositFromUserAllowed`, `depositIntoAllocatorAllowed`, `swapInputTokenAllowed`, `swapOutputTokenAllowed`). It also maintains a trusted set, which is distinct from being registered. Only trusted assets contribute to balance aggregation and are eligible for user deposits or swap outputs. Calling `distrustAsset()` automatically disables deposits and swap output for the asset. All enabling operations have a 7-day delay, whereas disabling operations are immediate. Assets are limited to 18 decimals for RAY precision compatibility.

`WithdrawalExecutionPolicy` enforces a fee on withdrawals (both `StableVault.executeWithdrawal()` and `EarningChainGateway.exchangeIouTokens()`). The base fee is the asset-specific fee if one is configured for the output asset, or the global default fee otherwise. These two do not overwrite each other. If the caller supplies a valid EIP-712 signed personal fee from a whitelisted signer and that personal fee amount is lower than the base fee, it overrides the base fee. All fees are capped at 10% (1000 bps). The signed discount mechanism uses per-signer nonces with deadlines. The contract also maintains a global redemption bucket that rate-limits the total volume of withdrawals processed per time window.

`DepositPolicy` enforces deposit rate limits at two levels: a per-asset bucket for each registered asset, and a global bucket that caps total deposit volume across all assets with amounts normalized to 18 decimals. Deposits that would exceed either bucket are rejected. A zero-capacity bucket (the default) blocks all deposits until the operator configures it.

`FundsBridgingPolicy` enforces rate limits on cross-chain fund transfers at two levels: a per-route bucket keyed by (`asset`, `destinationChainId`, `bridgeAdapter`) triple, and a global bucket that caps total bridging volume across all routes with amounts normalized to 18 decimals. This limits the volume of assets that managers can bridge over any given time window, bounding the exposure from a compromised manager role.

`PolicyRegistry` is a simple key-value store mapping bytes32 policy IDs to policy contract addresses. It is the resolution point for all policy lookups in `StableVault`, `FundsHandler`, `EarningChainGateway`, and `Allocator`. Setting a policy address to zero effectively disables the policy check for that ID. Changes to the registry are subject to a 14-day critical delay.

`Swapper` is owned by the `Allocator` and executes arbitrary swap logic during rebalancing via an array of calls to external targets. It enforces a 1:1 conversion between input and output assets (adjusted for decimals). The slippage tolerance is bounded by the `SlippageCoverageVault`'s configured `maxSlippageBps` (or `overrideMaxSlippageBps` in override mode). Shortfalls within the tolerance are covered by pulling from the `SlippageCoverageVault`. Surplus input is reimbursed back to the vault.

SlippageCoverageVault holds capital provided by governance to cover slippage shortfalls. It enforces two independent caps on coverage disbursements: a per-operation cap and a fixed-window rolling cap per asset. An override mode bypasses both caps and applies a separate slippage tolerance, intended for emergency rebalancing situations. The vault can be funded and swept by governance.

TransferHelper is a stateless, permissionless intermediary that holds assets temporarily during multi-step flows, enabling non-adjacent contracts to transfer assets without direct approvals. The modifier `assertingTransferHelperBalanceFor` in `TransferHelperClient` is used to ensure that the helper's balance does not increase by the end of the transaction.

OwnedMulticall is a privileged batching proxy that lets the owner execute multiple protocol actions atomically. To this end, the deployed contract will hold protocol roles. The owner has unrestricted ability to call any address with any calldata through the aggregate functions. The contract can also act as the slippage coverage source for Swapper. The functionality to renounce the ownership is permanently disabled.

2.4 Changes in Version 2

The following changes were made in **Version 2** of the contracts:

- **Split bridge-adapter registries in BaseChainGateway.** The single `addBridgeAdapter` / `removeBridgeAdapter` pair was replaced by separate funds and data-only registries: `addFundsBridgeAdapter` / `removeFundsBridgeAdapter` (per asset and chain) and `addDataOnlyBridgeAdapter` plus a two-step, timelocked data-only removal (`initiateDataOnlyBridgeAdapterRemoval` and `finalizeDataOnlyBridgeAdapterRemoval`) using a per-adapter mode (`SEND_AND_RECEIVE` / `RECEIVE_ONLY`) and a `removalId`. The last send-enabled data-only adapter for a chain can no longer be removed.
- **StableVault.balanceOf() floors at the original deposit.** `_getUserBalance` now returns `max(shares * conversionRate, originalDepositRay)`, and `totalSupply()` (active sub-vault obligations) is likewise floored at the global sum of original deposits. This also tightens the `claimSurplusInterest()` solvency check, which can no longer under-report obligations below the principal owed to users, see [claimSurplusInterest\(\) can underreport the vault's obligations](#).
- **StableVault.transfer() caps at the share-backed value.** A partial transfer is now bounded by `shares * conversionRate` (no original-deposit floor). Recipient shares are recomputed upon a same sub-vault transfer instead of reusing the sender's burnt shares.
- **WithdrawalExecutionPolicy allows zero redemption-bucket floors.** The constructor no longer requires non-zero `minRedemptionCapacity` / `minRedemptionRefillRate`. Zero floors let admins halt redemptions via `lowerRedemptionCapacity(0)` / `lowerRedemptionRefillRate(0)`.

2.5 Changes in Version 4

The following changes were made in **Version 4** of the contracts:

- **WithdrawalExecutionPolicy made non-upgradeable.**
- **Global rate-limit buckets added to DepositPolicy and FundsBridgingPolicy.** The new `GlobalRateLimitedPolicy` base contract provides a single token-bucket that bounds total throughput across all per-key buckets, with amounts normalized to 18 decimals. `DepositPolicy` gains a global cap across all assets (`raiseGlobalDepositCapacity`, `lowerGlobalDepositCapacity`, `raiseGlobalDepositRefillRate`, `lowerGlobalDepositRefillRate`); `FundsBridgingPolicy` gains a global cap across all (asset, chain, adapter) routes (`raiseGlobalBridgingCapacity`, `lowerGlobalBridgingCapacity`, `raiseGlobalBridgingRefillRate`, `lowerGlobalBridgingRefillRate`).

2.6 Trust Model

Operator: Users deposit assets into a managed vault where operators control strategy selection, cross-chain fund distribution, and interest rate assignments. Operators can move users to different sub-vaults without delay, affecting future interest accrual. Sub-vault rates can be changed with a 4-hour delay. While the system provides safeguards (timelocked administrative actions for strategy and adapter changes, guaranteed principal accounting that limits interest claims to available surplus, ordered redemption, and cross-chain IOU bridging to access liquidity on other chains), these protections rely on underlying liquidity being available. In case of losses from strategy failures or asset depeg, users are implicitly relying on operators to restore solvency.

Governance (AccessManager admin / High-Threshold Multisig): Privileged, but the system is designed to be non-custodial: every action capable of misappropriating user funds is subject to an execution delay, giving users a monitoring and exit window before the action takes effect. Within that constraint, there are delayed pathways through which a malicious governance could ultimately divert funds:

1. `claimSurplusInterest()` after manipulating balance oracle adapters to inflate `totalAssets`. (7-day delay)
2. Upgrading contracts to malicious implementations. (all core contracts are upgradeable via proxy, 14-day delay)
3. Adding a malicious strategy that siphons deposited assets. (7-day delay)
4. Adding a malicious bridge adapter that intercepts bridged funds. (7-day delay)

The AccessManager uses a two-tier guardian hierarchy (`ADMIN_ROLE_GUARDIAN_ROLE` and `OPERATIONAL_ROLE_GUARDIAN_ROLE`) that enables emergency role revocation without privilege escalation. Execution delays are organized into five levels: critical delay (14 days) for AccessManager admin operations, proxy upgrades, and `setPolicy`; high delay (7 days) for sensitive configuration changes enabling fund movement (strategies, bridges, oracles, enabling deposits, treasury, surplus claims, and raising rate limits); medium delay (1 day) for setting the default sub-vault; low delay (4 hours) for setting sub-vault rates; and no delay for operational and emergency functions.

Manager (Rebalancer role): Partially trusted. Can execute `rebalance()` on the Allocator which includes arbitrary external calls via the Swapper (to DEX routers etc.). Swaps enforce 1:1 conversion with slippage bounds enforced by `SlippageCoverageVault`, and only whitelisted swap-eligible assets can be swapped. Can call `pushFundsToChain()` and `pushFundsToAccountingChain()` to bridge assets between chains, subject to `FundsBridgingPolicy` rate limits, but only to registered Earning Chains and through whitelisted bridge adapters. Cannot directly withdraw assets to arbitrary addresses. A compromised manager could cause temporary liquidity shortages by misallocating funds across chains or concentrating assets in underperforming strategies, up to the bounds set by the bridging and rebalance policies.

Withdrawal Fee Signers: Partially trusted. Can sign EIP-712 fee discount messages that reduce or eliminate the withdrawal fee for specific users. Cannot steal funds directly. A compromised signer can issue zero-fee withdrawals, eliminating the protocol's fee revenue and enabling costless arbitrage against the vault.

Policy Configurators: Partially trusted. Can raise or lower token-bucket capacity and refill rates for deposit, withdrawal, and bridging policies. Raising capacity or refill rates beyond safe bounds could allow a manager to bridge excessive funds or flood the vault with deposits from a compromised asset. Lowering capacity to zero effectively halts the corresponding operation.

End Users: Untrusted. Deposits are gated by AssetRegistry whitelisting, oracle price checks, and DepositPolicy rate limits. Withdrawals enforce fees and solvency checks. IOU bridging uses lock-on-source/mint-on-destination with finality assumptions. Users can specify an arbitrary receiver for `exchangeIouTokens()` on Earning Chains, but this only directs their own withdrawn assets. Users pay bridge fees and cannot grief the protocol without economic cost (withdrawal fees, gas). Users are expected to be aware of the system's non-custodial design: privileged actions that could affect the

system's configuration and hence access to funds are gated by execution delays, and users are expected to withdraw within those exit windows in case of unexpected or unfavorable changes.

Chainlink Oracles (Price + Balance): The system relies on Chainlink price feeds for deposit gating, balance valuation, and swap validation. A manipulated or stale price feed could:

1. Allow deposits of a depegged asset at inflated value.
2. Undervalue assets causing the system to appear insolvent and block interest withdrawals.

The Chainlink balance oracle relays Earning Chain balances to the Accounting Chain. A stale balance feed causes the system to treat the Earning Chain balance as zero, underestimating total assets.

CCIP Bridge: The system relies on Chainlink CCIP for cross-chain message and token delivery for fund-bearing messages. Message authentication is enforced by validating the source chain adapter address against the registered `_destinationChainAdapterOf` mapping. A failure or compromise of CCIP could:

1. Delay or prevent cross-chain fund movements and IOU bridging.
2. If message forgery were possible (requires CCIP-level compromise), allow minting of unbacked IOUs on Earning Chains or burning of IOUs without corresponding asset release.

a.DI Bridge: The system uses a.DI for data-only cross-chain messages (BURN_IOU_TOKEN and BRIDGE_IOU_TOKEN). Replay protection is enforced by a.DI on the envelope level. A compromised a.DI could allow forgery of

- BURN_IOU_TOKEN messages: burning locked IOUs without a corresponding exchange on the Earning Chain, reducing the system's (guaranteed) obligations
- BRIDGE_IOU_TOKEN messages: minting unbacked IOUs on the Earning Chain, or releasing IOUs to an unintended recipient on the Accounting Chain. In the latter case, legitimate users may be unable to redeem their full principal unless the vault has accumulated sufficient surplus interest.

ERC-4626 Strategies: Trusted. Each strategy is whitelisted with a 7-day delay. The deposit path tolerates only up to `STRATEGY_DEPOSIT_SLIPPAGE_TOLERANCE` (a fixed global constant, in the asset's smallest unit) of difference between the deposited amount and the value of the shares received; withdrawals from a strategy must deliver the full requested amount or revert. A malicious or buggy strategy can cause fund loss. The system assumes:

- Strategies are audited and vetted before whitelisting (7-day delay)
- Strategies do not charge withdrawal fees (required for accurate `previewRedeem()` accounting)
- Strategies are not susceptible to inflation attacks
- Off-chain monitoring continuously tracks strategy health and reduces exposure if risk tolerance is breached
- If a strategy is compromised, deposits are disabled immediately and the strategy can be distrusted to exclude it from solvency accounting
- Strategies do not deposit into StableVault

If a strategy violates these assumptions, the system can lose funds. Inflation attacks are particularly dangerous at deposit time: if a strategy is inflated when assets are deposited, the system receives fewer shares than the real value while crediting users with full `originalDepositRay`. Beyond inflation, a strategy could lose deposited funds entirely through DeFi risk.

Upgradeability: All core contracts (StableVault, FundsHandler, Allocator, AccountingChainGateway, EarningChainGateway, AssetRegistry, PriceOracle, ChainBalanceOracle) are upgradeable via `AccessManagedUpgradeable`. Bridge adapters (CcipAdapter, AdiAdapter) and utility contracts (Swapper, TransferHelper, OwnedMulticall, IouToken, IouTokenManager, SlippageCoverageVault, PolicyRegistry, DepositPolicy, FundsBridgingPolicy, WithdrawalExecutionPolicy) are not upgradeable. The upgrade

authority rests with the AccessManager admin, subject to the critical delay (14 days). Once executed, a malicious upgrade can bypass all system invariants.

2.6.1 Operational Assumptions

Malicious or compromised manager accounts are expected to be revoked within 1 hour of detection via guardian action, enabled by off-chain monitoring. The system assumes correct cross-chain configuration: oracle adapters, bridge adapters, and chain registrations must be configured consistently across all chains. Policy buckets (deposit, bridging, redemption) are assumed to be configured conservatively. Misconfigured high-capacity buckets reduce the protection they provide.

2.6.2 Appendix: Roles Setup

This appendix documents the roles configuration of the system.

The system uses OpenZeppelin's AccessManager as the central access control registry, deployed independently on each chain. All contracts inheriting AccessManaged reference their chain's manager, which maps function selectors and targets to required roles and enforces execution delays.

Role Hierarchy

Three special roles govern access control:

- **ADMIN_ROLE**: Super admin with 14-day execution delay. Can call any AccessManager function including proxy upgrades.
- **ADMIN_ROLE_GUARDIAN_ROLE**: Can grant and revoke admin-tier function roles. Grants are subject to each role's grant delay, which equals the function's execution delay. Revocations are immediate.
- **OPERATIONAL_ROLE_GUARDIAN_ROLE**: Can grant/revoke operational function roles (immediate, non-critical).

The guardian roles enable emergency revocation.

Function Roles

The system defines function-based roles derived from selectors. Each specifies an execution delay and guardian assignment. The following functions are marked *critical-risk* (can directly compromise funds): `setAssetConfig`, `addFundsBridgeAdapter`, `addDataOnlyBridgeAdapter`, `initiateDataOnlyBridgeAdapterRemoval`, `finalizeDataOnlyBridgeAdapterRemoval`, `setDestinationChainAdapter`, `setChainSelector`, `setTreasury`, `addStrategy`, `setChainBalanceOracleAdapter`, `setPolicy`, and the a.DI `CrossChainController` functions (`approveSenders`, `removeSenders`, `enableBridgeAdapters`, `disableBridgeAdapters`, `updateOptimalBandwidthByChain`, `configAdapter`, `updateRequiredForwardingSuccessesByChain`, `updateConfirmations`, `updateMessagesValidityTimestamp`, `allowReceiverBridgeAdapters`, `disallowReceiverBridgeAdapters`, `emergencyTokenTransfer`, `emergencyEtherTransfer`, `transferOwnership`, `updateGuardian`). Note that *critical-risk* classifies the fund impact of a function and is independent of its execution-delay tier: the system contract functions among them sit in the high (7-day) tier, while `setPolicy` and the a.DI `CrossChainController` functions (and the **ADMIN_ROLE** operations and proxy upgrades) sit in the critical (14-day) tier. The rate-limit raise functions, `SlippageCoverageVault` configuration functions (`raiseDepositCapacity`, `raiseDepositRefillRate`, `raiseGlobalDepositCapacity`, `raiseGlobalDepositRefillRate`, `raiseBridgingCapacity`, `raiseBridgingRefillRate`, `raiseGlobalBridgingCapacity`, `raiseGlobalBridgingRefillRate`, `raiseRedemptionCapacity`, `raiseRedemptionRefillRate`, `raisePullCapPerTx`, `raiseWindowCap`, `lowerWindowSeconds`, `setMaxSlippageBps`, `setOverrideMaxSlippageBps`), and the sweep function sit in the high tier but are not classified critical-risk.

Delay Tiers



- *Critical (14 days)*: ADMIN_ROLE operations, proxy upgrades, setPolicy, and all a.DI CrossChainController functions.
- *High (7 days)*: Configuration changes enabling fund movement (adding strategies, bridges, oracles, enabling deposits, trusting assets/strategies, setting the treasury, claiming surplus interest, adding signers, and raising rate-limit buckets).
- *Medium (1 day)*: Setting the default sub-vault.
- *Low (4 hours)*: Setting sub-vault rates.
- *Immediate*: Operational and emergency functions (rebalancing, disabling, distrusting, removing, rescue, lowering rate-limit buckets, and override-mode toggles).

Disabling operations are immediate for emergency response.

Profiles

Profiles are predefined bundles of roles.

- *MainAdmin*: ADMIN_ROLE + both guardian roles + all function roles. Full control with delays.
- *SecondaryAdmin*: OPERATIONAL_ROLE_GUARDIAN_ROLE + all non-critical function roles. Cannot call critical-risk functions such as adding strategies, bridges, setting the treasury or policies, or a.DI CrossChainController functions. It can raise rate-limit buckets and set SlippageCoverageVault parameters, since those are not critical-risk.
- *StableVaultManager* (Accounting Chain only): Interest rate management functions.
- *WithdrawalPolicyManager*: Fee configuration (setDefaultFeeBps, setAssetFeeBps).
- *Rebalancer*: Fund movement and rebalancing (rebalance(), pushFundsToChain(), pushFundsToAccountingChain(), setWithdrawalQueue(), disableDepositsToStrategy()).
- *Disabler*: Emergency shutdown functions. Rebalance assets, remove, disable, distrust, and rate-limit lowering functions (lowerDepositCapacity, lowerDepositRefillRate, lowerGlobalDepositCapacity, lowerGlobalDepositRefillRate, lowerBridgingCapacity, lowerBridgingRefillRate, lowerGlobalBridgingCapacity, lowerGlobalBridgingRefillRate, lowerPullCapPerTx, lowerWindowCap, raiseWindowSeconds). Redemption rate-limit lowering is not granted to this profile, and rescue is a separate profile.
- *Funder*: Liquidity top-up (topUp(), fundCoverage()).
- *ATokenVaultRewardClaimer*: Merkl reward claiming and emergency rescue on aToken vaults.
- *Rescuer*: Token and native rescue (rescueTokens(), rescueNative()).

Rebalancer and Disabler operate through OwnedMulticall contracts for atomic batching.

Proxy Upgrades

All upgradeable contracts use TransparentUpgradeableProxy with proxy admin owned by AccessManager. Upgrades require ADMIN_ROLE (14-day delay).

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the [Resolved Findings](#) section. The findings are split into these different categories:

- **Design**: Architectural shortcomings and design inefficiencies
- **Correctness**: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	1

- [Users Can Cause setUserRate Batch to Revert](#) **Acknowledged**

5.1 Users Can Cause setUserRate Batch to Revert

Design **Low** **Version 1** **Acknowledged**

CS-ASV2-001

When the admin prepares a batch to reset user rates, the intended new rate for a user may already equal the rate of the sub-vault to which the user belongs at the time of transaction execution. This can happen when a user, who is assigned to a sub-vault different from the default one, exits the vault (clearing their position) and then re-enters before the batch is executed. On re-entry, the user is assigned to the default sub-vault. If the default sub-vault's rate matches the rate the admin intended to assign in the batch, the entire transaction reverts.

Acknowledged:

Aave Labs acknowledged the griefing surface and commented:

The on-chain default sub-vault has a perSecondRate of 1 RAY, so it may be lower than the off-chain default sub-vault. If an admin ever assigns a user to the on-chain default sub-vault, which is expected to be rare, that assignment will be handled in isolation.

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the [Open Findings](#) section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	3

- Disabler Missing Roles in WithdrawalExecutionPolicy **Specification Changed**
- CcipAdapter.setChainSelector() Leaves Dangling Mappings **Code Corrected**
- balanceOf() Does Not Reflect the Principal Preservation Guarantee, Breaking the transfer(to, balanceOf(user)) Pattern **Code Corrected**

Informational Findings	9
------------------------	---

- FundsHandler.addEarningChain() Does Not Require a Configured Chain Balance Oracle Adapter **Code Corrected**
- Inaccurate NatSpec for Coverage Flows in SlippageCoverageVault **Specification Changed**
- IouTokenManager Inherits TransferHelperClient but Never Uses It **Code Corrected**
- Missing Authority Validation in SlippageCoverageVault Constructor **Code Corrected**
- Redemption Bucket Not Seeded at Deployment **Code Corrected**
- _isStrategySupportedForAsset Returns True for (address(0), address(0)) **Code Corrected**
- claimSurplusInterest() Can Underreport the Vault's Obligations **Code Corrected**
- overrideMaxSlippageBps Not Enforced to Exceed _maxSlippageBps **Specification Changed**
- addEarningChain Accepts chainId == 0 Unlike the Gateway's Bridge Adapter Whitelist **Code Corrected**

6.1 Disabler Missing Roles in WithdrawalExecutionPolicy

Correctness **Low** **Version 3** **Specification Changed**

CS-ASV2-016

The Stable Vault Roles Spreadsheet lists the Disabler profile able to call lowerRedemptionRefillRate() and lowerRedemptionCapacity() on the WithdrawalExecutionPolicy.

_setupProfile_Disabler() in AccessManagerBaseSetup.sol however does not grant these capacities; it only grants removeSigner() for the Disabler prprofile.

Specification changed:



The Stable Vault Roles Spreadsheet has been updated to remove the Disabler profile from `lowerRedemptionRefillRate()` and `lowerRedemptionCapacity()`.

6.2 CcipAdapter.setChainSelector() Leaves Dangling Mappings

Design Low Version 1 Code Corrected

CS-ASV2-010

An admin reassigning a `chainId` to a new CCIP selector, or a CCIP selector to a new `chainId`, leaves stale entries in the mirrored mapping. Subsequent outbound calls through `CcipAdapter.publishMessageWithFunds()` can then route bridge fund pushes to the wrong destination, because the outbound path uses `_chainSelectorOf[targetChainId]` without any consistency check against the reverse mapping.

The two mappings are intended to form a bidirectional translation between EVM chain identifiers and CCIP-specific selectors:

```
mapping(uint256 chainId => uint64 ccipChainSelector) internal _chainSelectorOf;
mapping(uint64 ccipChainSelector => uint256 chainId) internal _chainIdOf;
```

`CcipAdapter.setChainSelector()` writes both mappings unconditionally, but it does not read any previous pairing or clear the stale reverse entry on reassignment:

```
function setChainSelector(uint256 chainId, uint64 ccipChainSelector) external override restricted {
    _chainSelectorOf[chainId] = ccipChainSelector;
    _chainIdOf[ccipChainSelector] = chainId;
    emit ChainSelectorSet(chainId, ccipChainSelector);
}
```

After `setChainSelector(X, A)` followed by `setChainSelector(X, B)`, the entry `_chainIdOf[A] = X` from the first call remains. After `setChainSelector(X, A)` followed by `setChainSelector(Y, A)`, the entry `_chainSelectorOf[X] = A` from the first call remains while `_chainIdOf[A] = Y` overwrites the previous reverse direction. In both shapes the two mappings disagree about the current pairing.

On the inbound path, `CcipAdapter._validateMessageSource()` performs a bidirectional cross-check:

```
uint256 chainIdFromMessageChainSelector = _chainIdOf[message.sourceChainSelector];
address destinationChainAdapter = _destinationChainAdapterOf[chainIdFromMessageChainSelector];
require(destinationChainAdapter != address(0), Errors.InvalidParameter());
require(
    message.sourceChainSelector == _chainSelectorOf[chainIdFromMessageChainSelector],
    Errors.InvalidParameter()
);
```

This rejects any incoming message whose `sourceChainSelector` resolves to a `chainId` whose own `_chainSelectorOf` entry no longer matches that selector. Dangling reverse-direction entries from reassignment therefore cannot be exploited to deliver forged messages. The worst outcome on inbound messages is that legitimate messages with the stale selector are rejected.

The outbound path has no equivalent check. `CcipAdapter.publishMessageWithFunds()` and `CcipAdapter._publishCcipMessage()` resolve `_chainSelectorOf[targetChainId]` and pass it directly to `IRouterClient.getFee()` and `IRouterClient.ccipSend()` via the assembled `Client.EVM2AnyMessage`. If admin reassignment has produced two `chainId` values that both map to the same selector, manager-issued `FundsHandler.pushFundsToChain()` calls targeting either of

those `chainId` values resolve to that single selector and CCIP delivers the funds to whichever chain it associates with that selector, regardless of which logical `targetChainId` was passed.

Additionally, `CcipAdapter.setChainSelector()` accepts zero for either argument without restriction, even though both mappings rely on 0 as the implicit "unconfigured" sentinel. Misconfiguration would collapse the sentinel into a real chain, but realistic exploitation requires a CCIP source chain that uses selector 0, which is not the case in practice.

Code corrected:

The function `setChainSelector` has been updated to clean up the reverse mapping upon an update.

6.3 `balanceOf()` Does Not Reflect the Principal Preservation Guarantee, Breaking the `transfer(to, balanceOf(user))` Pattern

Correctness **Low** **Version 1** **Code Corrected**

CS-ASV2-015

The function `_getUserBalance()` (used by `balanceOf()` and `getUserBalance()`) returns `shares.rayMulDown(conversionRate)`, the value of the user's shares at the current conversion rate (rounded down). When this falls below `originalDepositRay` (i.e. when the share derived value alone does not cover the original deposit), the principal preservation floor in `_previewFullWithdrawalRequest()` bumps the user's full withdrawal entitlement up to `originalDepositRay`. `balanceOf()` does not apply this floor. It keeps reporting the lower value even though the user's actual entitlement on full withdrawal is `originalDepositRay`.

Calling `transfer(to, balanceOf(user))` for such a user reverts. The transfer takes the partial transfer path in `_computeTransferShares()` (because the requested amount is below the bumped `fullAmountRay`), then burns all of the sender's shares and trips the `_areRemainingSharesRedeemable` check with `Errors.InvalidAmount()`. Users following the natural ERC20 pattern `transfer(to, balanceOf(user))` must instead call `transferAll(to)`.

Code corrected:

The function `_getUserBalance()` has been updated to return `originalDepositRay` if the value of the user's shares at the current conversion rate falls below it.

Calling `transferTo(to, balanceOf(user))` can still revert as documented by Aave Labs.

```
uint256 fromUserTotalShares = $storage().positions[from].shares;
uint256 totalAvailableAmountRayFromShares = fromUserTotalShares.rayMulDown(fromConversionRate);

// If the amountRay is greater than the total available amount from shares, the user should use transferAll()
// instead.
require(amountRay <= totalAvailableAmountRayFromShares, Errors.InvalidAmount());
```

6.4 FundsHandler.addEarningChain() Does Not Require a Configured Chain Balance Oracle Adapter

Informational

Version 1

Code Corrected

CS-ASV2-011

`FundsHandler.addEarningChain()` allows registering an Earning Chain without checking that the `ChainBalanceOracle` has an adapter configured for it. While a registered chain has no adapter, `FundsHandler.getAggregatedBalance()` reverts, and every path that reads it reverts with it.

The cause is an asymmetry in how a missing adapter and a stale adapter are handled. `ChainBalanceOracle.getChainBalance()` enforces `adapter != address(0)` outside its surrounding try-catch, so an unconfigured chain reverts rather than taking the stale-data fallback that an adapter-call failure would. `FundsHandler._getAdjustedEarningChainBalanceRay()` calls `getChainBalance()` with no try-catch of its own, so the revert propagates through the per-chain loop in `getAggregatedBalance()`. A configured-but-stale adapter, by contrast, returns `isStale = true` and the chain contributes 0.

`StableVault.requestWithdrawal()` reads the aggregated balance unconditionally at the start of its solvency check, so while a registered chain lacks an adapter the call reverts for every user and every amount. This includes principal-only requests: the aggregated balance is only needed to bound the interest portion of a request against available surplus, and a principal-only request carries no interest portion, but the read happens before that gate and is not conditional on it, so the request reverts before reaching the logic that would have passed it trivially. `StableVault.claimSurplusInterest()`, the aggregated-balance views, and `AccountingChainGateway.sendPushFundsToChainMessage()` for the misconfigured chain revert for the same reason. `deposit()` and `executeWithdrawal()` are unaffected, since neither reads the aggregated balance.

Code corrected:

The function `FundsHandler.addEarningChain(chainId)` has been updated to invoke the `ChainBalanceOracle.getChainBalance(chainId)`. When `ChainBalanceOracle` does not have a configured adapter for `chainId`, the call will revert and prevent an admin from registering an Earning Chain without a configured adapter.

6.5 Inaccurate NatSpec for Coverage Flows in SlippageCoverageVault

Informational

Version 1

Specification Changed

CS-ASV2-002

The contract-level NatSpec of `SlippageCoverageVault` describes coverage flows as "Push-based flows to/from the immutable bound Swapper". This is inaccurate for inflows. The function `reimburseCoverage` transfers tokens from the Swapper via `safeTransferFrom`, which requires the Swapper to have approved `SlippageCoverageVault` as a spender beforehand.

Specification changed:

The natspec has been updated.

6.6 IouTokenManager Inherits TransferHelperClient but Never Uses It

Informational Version 1 Code Corrected

CS-ASV2-009

IouTokenManager inherits TransferHelperClient but no function references it. Prior to the bridge adapter refactor, `IouTokenManager.bridgeTokens()` was decorated with `assertingTransferHelperBalanceFor(bridgeParams.feeToken)`, which required the inheritance.

Code corrected:

The legacy inheritance has been removed.

6.7 Missing Authority Validation in SlippageCoverageVault Constructor

Informational Version 1 Code Corrected

CS-ASV2-003

The SlippageCoverageVault constructor does not validate the authority parameter. Every other contract in the codebase calls `IAccessManager(authority).canCall(address(0), address(0), bytes4(0))` to verify that the provided address is a valid IAccessManager. This check is absent in SlippageCoverageVault. Additionally, no zero-address check is performed on authority, leaving this input completely unsanitized.

Code corrected:

The absent check has been added.

6.8 Redemption Bucket Not Seeded at Deployment

Informational Version 1 Code Corrected

CS-ASV2-006

The WithdrawalExecutionPolicy constructor stores `MIN_REDEMPTION_CAPACITY` and `MIN_REDEMPTION_REFILL_RATE` as immutables but does not initialize the redemption bucket with these values. The bucket starts at a capacity and refill rate of zero. This has the following effects:

- If the policy is registered in PolicyRegistry before the bucket is configured via `raiseRedemptionCapacity()` and `raiseRedemptionRefillRate()`, every withdrawal attempt reverts, blocking users from exiting the system and breaking the desired non-custodial property.
- Together with the fact that the two raise functions apply no minimum checks, an admin can set values below the minimums after deployment. The minimum limit is only enforced once an admin raises the capacity / refill rate above these minimum limits.

Code corrected:

In **Version 3**, both `raiseRedemptionCapacity()` and `raiseRedemptionRefillRate()` enforce the immutable floors: raising to a value below `MIN_REDEMPTION_CAPACITY / MIN_REDEMPTION_REFILL_RATE` now reverts with `BelowMinRedemptionCapacity / BelowMinRedemptionRefillRate`, matching the checks already present in the lower functions. The minimums are therefore enforced on every configuration change, not only once the bucket has been raised above them.

The redemption bucket itself still initializes at zero capacity and zero refill rate. The immutable minimums may deliberately be zero so that admins retain an emergency halt (documented in the constructor), so seeding the bucket from the minimums would not guarantee a non-zero throughput in any case.

6.9 `_isStrategySupportedForAsset` Returns `True` for `(address(0), address(0))`

Informational **Version 1** **Code Corrected**

CS-ASV2-014

`_isStrategySupportedForAsset(strategy, asset)` returns `$storage().strategyConfigs[strategy].asset == asset`. For an unconfigured strategy the stored asset defaults to `address(0)`, so calling it with `strategy = address(0)` and `asset = address(0)` returns `true`. This check is the sole whitelist gate inside `_deallocate` and `_allocate`. A rebalance carrying `DeallocationParams/AllocationParams` with both fields set to zero therefore passes the gate and proceeds to call `IERC4626(address(0))` methods that revert deeper, masking the failure as a low-level revert rather than the intended `AddressNotWhitelisted` selector.

Code corrected:

The internal function has been modified to return `false` if `asset == address(0)`.

6.10 `claimSurplusInterest()` Can Underreport the Vault's Obligations

Informational **Version 1** **Code Corrected**

CS-ASV2-013

`claimSurplusInterest()` must ensure that all users' principal and interest can still be served before the treasury is allowed to take the surplus. It uses `_getVaultObligations() = _getActiveSubVaultsObligations() + _getIousInCirculation()` for this check, where `_getActiveSubVaultsObligations()` reconstructs principal as $\sum_v \text{totalShares}_v \cdot \text{rayMulUp}(\text{rate}_v)$.

`deposit()` rounds shares down via `rayDivDown` while crediting the full deposit amount to `globalOriginalDepositsRay`. When the conversion rate at deposit time exceeds `RAY` (i.e. interest has already accrued in the subvault), `rayMulUp(shares, conversionRate)` falls short of the original deposit by $\text{floor}((\text{rate} - 1) / \text{RAY})$ wei in `RAY` scale. Aggregated across users in a subvault, the share-derived reconstruction can sit below the sum of original deposits credited at deposit time. While that gap holds, `_getVaultObligations()` underreports the true obligation and

`claimSurplusInterest()` permits a treasury claim that draws total assets below the level required to cover every user's original deposit.

After such a claim, the principal floor inside `_previewFullWithdrawalRequest()` still authorises a full deposit withdrawal for affected users: the interest portion check passes trivially at zero, and the withdrawal amount stays at `originalDepositRay`. If every affected user attempts a full `executeWithdrawal()` simultaneously, the cumulative principal disbursement exceeds the reduced asset pool and the last redeemer's call reverts inside `Allocator.withdraw()` with `InsufficientFunds`.

In practice the issue is primarily theoretical. The aggregate gap is bounded by a handful of wei in RAY scale per affected subvault, and interest accrual closes it monotonically over time. Reaching the revert further requires asset granularity to align: `claimSurplusInterest()` checks `_getVaultObligations() <= _getVaultAggregatedBalance()` in RAY scale, but treasury claims are denominated in asset decimals. For a 6 decimal stablecoin one wei of asset already corresponds to 10^{21} RAY, far above the gap, so no nonzero claim passes the check. The bug is only practically reachable when the treasury claims an asset whose smallest unit (in RAY) is comparable to the gap, e.g. an 18 decimal asset where one wei equals 10^9 RAY.

Code corrected:

`_getActiveSubVaultsObligations()` now returns the maximum of the share-based reconstruction and `globalOriginalDepositsRay`.

6.11 `overrideMaxSlippageBps` Not Enforced to Exceed `_maxSlippageBps`

Informational Version 1 Specification Changed

CS-ASV2-008

The `enableOverrideMode` `NatSpec` describes the Swapper as accepting "the higher `overrideMaxSlippageBps`," hinting that `_overrideMaxSlippageBps` should exceed `_maxSlippageBps`. Neither the constructor nor the setters enforce this. Note that this is not a functional issue: override mode's primary purpose is to bypass the per-transaction and window caps on `pullCoverage`, the slippage value is an independent value set by the operator. An operator may legitimately want cap bypass without slippage relaxation.

Specification changed:

The `NatSpec` has been updated to reflect that `_overrideMaxSlippageBps` is independent from `_maxSlippageBps`.

6.12 `addEarningChain` Accepts `chainId == 0` Unlike the Gateway's Bridge Adapter Whitelist

Informational Version 1 Code Corrected

CS-ASV2-012

`FundsHandler.addEarningChain()` only rejects `chainId == block.chainid` and duplicates. It does not reject `chainId == 0`. By contrast, `BaseChainGateway.addBridgeAdapter()` explicitly enforces `chainId != block.chainid && chainId != 0`.

Code corrected:

The missing check has been added.

7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

7.1 Missing Zero-Length Message Check Before `receiveMessage` Call

Informational **Version 1** **Acknowledged**

CS-ASV2-004

The `AdiAdapter.receiveCrossChainMessage()` does not verify that `message.length > 0` before invoking `receiveMessage()` on the gateway. When `message.length` is zero, `receiveMessage()` is a no-op, meaning the call succeeds silently without any meaningful action being taken.

Acknowledged:

Client acknowledged the issue and provided the following explanation:

We confirm that a zero-length message results in a silent no-op, but this is unreachable under our trust model: the call is gated by `onlyCrossChainController` and `originSender == trustedOriginSender`, and the trusted remote adapter only ever emits non-empty ABI-encoded `CrossChainMessage` payloads. The no-op moves no funds, changes no state, and bypasses no access control, and it mirrors the intentional generic-router design shared with `CcipAdapter`.

7.2 No Minimum `receiverExecutionGasLimit` on Manager-Controlled Bridging Flows

Informational **Version 1** **Acknowledged**

CS-ASV2-005

The manager could supply a small `receiverExecutionGasLimit` when bridging funds between chains. This would cause the destination-chain message to fail on arrival, temporarily reducing the system's reported total assets until the message is successfully retried.

Acknowledged:

Aave Labs acknowledged the observation and provided the following reasoning:

Off-chain components will monitor for failed/missing ingestion of funds on the destination and execute a retry.

7.3 Withdrawal Queue Iterates Distrusted Strategies, Allowing a Single Malicious Strategy to Grief Withdrawals

Informational Version 1 Acknowledged

CS-ASV2-007

`withdraw()` iterates `withdrawalQueues[asset]` until enough funds have been retrieved, calling `tryWithdrawFromStrategy` on every registered strategy regardless of trust. Distrusting a strategy does not remove it from the queue, and `setWithdrawalQueue` enforces a strict permutation of the registered set, so the strategy can only be reordered. Reverts and short returns from the strategy are handled by the try/catch around `tryWithdrawFromStrategy`, but the gas forwarded to the strategy's external calls is not capped, so each call receives 63/64 of the remaining gas. A distrusted or broken strategy positioned early in the queue can consume nearly all of that, leaving the outer `withdraw()` call with only a small fraction of the original gas budget.

Unaware users will submit a transaction with insufficient headroom and run out of gas. A user aware of the problem can, if willing to absorb the additional gas cost, manually supply additional gas up to the per-transaction ceiling (32M on the Accounting Chain Arbitrum One and ~16M on the EarningChain Ethereum as of the writing of this report). This generally leaves enough headroom after the grieving call to complete the next strategy and the closing operations.

The manager can mitigate this by moving the offending strategy to the end of the queue, but until that happens withdrawals could be impaired despite liquidity being available in later strategies.

Note that removing such a strategy is also not guaranteed and requires two steps: `removeStrategy` requires both `isTrusted == true` (so the manager must first re-trust the strategy via `trustStrategy`, subject to the 7-day high delay) and `balanceOf(address(this)) == 0` (so the share balance must be fully drained first). Draining requires the strategy's `balanceOf`, `maxRedeem`, and `redeem` calls to not revert, and `redeem` to actually burn the supplied shares. If any of these conditions fail, `_deallocate` with `amount = 0` (the only path that accepts partial drain without a `>= amount` post-check) reverts and the strategy cannot be drained. Even after draining, the `balanceOf` call inside `_removeStrategy` is unguarded. A strategy that reverts on `balanceOf` cannot be removed regardless.

Acknowledged:

Aave Labs acknowledged the issue and commented:

```
Distrusting a strategy is intended to write the value in the strategy off to zero; it is not intended to block withdrawals from the strategy. Setting the withdrawal queue will not have an execution timelock, so in the event a strategy is consuming gas in a such a way which denies withdrawals from other strategies, then the admin will re-order the withdrawal queue.
```

8 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

8.1 Allocator Receiving Funds via Bridging May Revert

Note **Version 1**

When funds are bridged between the Accounting Chain and an Earning Chain in either direction, there is no validation of the asset on the source-chain callpath. If the asset is not accepted for deposits on the destination chain, the message reverts on arrival because `Allocator.deposit()` enforces this. Note that enforcing this check is not straightforward.

8.2 Oracle Price Validation Does Not Enforce an Upper Bound

Note **Version 1**

`PriceOracle.validatePrice()` only checks that the price feed is not stale and that the price is at or above `MIN_VALID_PRICE_RAY`. No upper bound is enforced.

When a supported stablecoin depegs above its nominal value, a user can deposit it and receive shares minted 1:1 at its nominal ray value. Another user who already has shares can immediately call `requestWithdrawal()` and redeem their shares for the same asset at the same 1:1 nominal rate.

A user voluntarily depositing an asset worth more than what they receive in exchange is in itself unusual. Small fluctuations around the nominal value are expected and normal, but a depegged price above a certain threshold, combined with such a deposit where the user seemingly gives up value, may not be as safe as expected but could be indicative of a problem with the stablecoin, its price oracle, or both, that the system does not detect.

8.3 Re-added Bridge Adapter Resumes Previous Rate-Limit Bucket State

Note **Version 1**

In `FundsBridgingPolicy`, the rate-limit bucket for a given (asset, destChainId, bridgeAdapter) triple is stored independently of the bridge adapter whitelist maintained in `BaseChainGateway`. Removing a bridge adapter via `removeBridgeAdapter()` does not reset or delete the corresponding bucket. Should an adapter be re-added, it goes through `addBridgeAdapter()` (high delay, 7 day timelock), so for any non-zero refill rate the bucket will have refilled over that window and a usable rate limit remains in place.

8.4 StableVault Exposes ERC20-like Functions Without Implementing ERC20

Note **Version 1**

StableVault is not an ERC20 token and does not implement IERC20, but it exposes several functions that share names and signatures with the ERC20 standard: `transfer()`, `totalSupply()`, `balanceOf()`, `name()`, `symbol()`, and `decimals()`. These functions operate on vault positions rather than a fungible token balance. Integrators or tooling that rely on interface detection should not incorrectly treat StableVault as an ERC20 token.

8.5 WithdrawalRequestPolicy Can Restrict User Exits From StableVault

Note **Version 1**

StableVault supports a configurable `WithdrawalRequestPolicy`. This breaks the non-custodial guarantee that users can always exit the vault and receive their IOU tokens. Note that the policy is not configured in the current deployment.

Setting a `WithdrawalRequestPolicy` requires calling `setPolicy` on the `PolicyRegistry`. In the intended deployment configuration, this function carries a critical execution delay of 14 days, so a newly set policy cannot take effect until 14 days after it is scheduled. During that window user exits remain subject only to the currently active configuration. Aave Labs intends this delay to let the public evaluate a pending policy change before it takes effect.